

Learning transferable collaborative behaviors for multiple agents in the game environment

Wei Li ^a, Jiali Lv ^a, Xu Zhang ^b, Kaizhu Huang ^c, Aiguo Song ^a

^a School of Instrument Science and Engineering, Southeast University, Nanjing, 210096, China

^b National Key Laboratory of Avionics Integration and Aviation System-of-Systems Synthesis, China Aeronautical Radio Electronics Research Institute, Shanghai, 200241, China

^c Digital Innovation Research Center, Duke Kunshan University, Kunshan, 215316, China

ARTICLE INFO

Keywords:

Cooperative multi-agent reinforcement learning
Policy learning
Policy transfer
Multi-agent systems
Real-time strategy game

ABSTRACT

Cooperative Multi-Agent Reinforcement Learning (CMARL) enables multiple agents to learn collaborative policies for accomplishing complex tasks in virtual game environments. However, most CMARL algorithms have difficulties in learning effective collaborative policies in single one scenario and transfer them across scenarios. To address these challenges around the multi-agent systems, we propose a novel method, named Collaborative Policy Learning and Transfer (CPLT). CPLT comprises two key components: the Control Action Generation (CAG) module, which learns the effective and transferable policies from the local observations of agents, and the Collaborative Control Optimization (CCO) module, which precisely evaluates the agent contributions to improve the teamwork. We evaluate CPLT on StarCraft Multi-Agent Challenge (SMAC), a benchmark platform of the real-time strategy game StarCraft II, and Multi-Agent Particle Environment (MPE), a two-dimensional platform for the cooperative and competitive tasks. Experimental results demonstrate that CPLT can effectively improve the agent performances in both within-scenario and cross-scenario collaborative tasks. Moreover, the CAG and CCO modules show strong compatibility with the related CMARL methods for enhancing their gaming performances.

1. Introduction

Many complex decision-making tasks involve the interaction and collaboration among multiple agents, such as swarm coordination [1, 2], autonomous driving [3,4], and gaming applications [5–7], all of which can be modeled as Multi-Agent Systems (MAS). Beyond these typical scenarios, the similar challenge for flexible multi-entity coordination also exists in broader systems, such as load balancing in cloud computing and task planning in distributed systems [8–10]. Cooperative Multi-Agent Reinforcement Learning (CMARL) offers a promising methodology to complete the collaborative tasks by integrating deep reinforcement learning with MAS [11]. This methodology is not only quite useful for agent collaboration, but also holds significant potential for task scheduling, resource allocation, and load balancing in complex systems [12–14]. Most CMARL methods follow the Centralized Training and Decentralized Execution (CTDE) paradigm [15]. In the decentralized execution phase, CMARL algorithms generate the policies based on the local observations of agents to control their actions. In the centralized training phase, CMARL algorithms optimize the policies of agents to enhance their collaborative performances. This learning paradigm is

particularly well-suited for real-time strategy games, where agents need to act autonomously with limited environmental information. CMARL methods can be categorized into two sorts, value-based and policy-based, according to their optimization objectives. The former focuses on optimizing an action-value network to improve agent behaviors, such as duPLEX dueling multi-agent Q-learning (QPLEX) [16] and Mixture of Categorical distributions for MARL (MCMARL) [17]. The latter directly optimizes the policies for controlling agent actions, such as Learning Joint-Action Intrinsic Reward (LJIR) [18] and Counterfactual Multi-Agent (COMA) [19]. These methods can control multiple agents to learn the collaborative behaviors in virtual game environments.

Value-based methods typically control the actions of agents based on an action-value network that estimates the individual Q -values from the local observations of agents. These individual Q -values are then combined into a joint Q -value through the mixing network to facilitate collaborative policy optimization [20,21]. For example, QMIX [22] utilizes Gated Recurrent Unit (GRU) [23] in the action-value network for decision-making and employs the hypernetworks to integrate the global state information for policy optimization. Universal

* Corresponding author.

E-mail address: li-wei@seu.edu.cn (W. Li).

<https://doi.org/10.1016/j.entcom.2025.101051>

Received 10 April 2025; Received in revised form 9 October 2025; Accepted 1 November 2025

Available online 10 November 2025

1875-9521/© 2025 Published by Elsevier B.V.

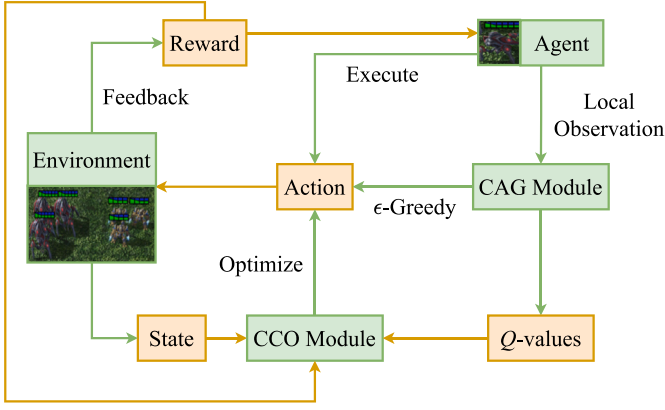


Fig. 1. The interaction between the agents and the environment in CPLT.

Policy Decoupling Transformer (UPDeT) [24] applies the Transformer to extract the spatial features and map them to the agent actions for decision-making. TransfQMix [25] models the multi-agent environment as a graph structure, and adopts Transformers in both action-value and mixing networks to perform graph-based reasoning for policy generation.

However, most action-value networks have not fully utilized the observation data to control agent behaviors in completing complex tasks [24,25]. Moreover, the mixing network may process the individual Q -values of inactive agents, thus probably misleading the contribution assessment of the agents in policy optimization [26]. Furthermore, variations in the composition of local observation, action set, and agent group across different scenarios usually lead to the dimensional differences of both input and output data of the CMARL models. The different dimensions of data straightforwardly prevent the CMARL models trained in one scenario from being transferred to another. To address these challenges, we propose a novel and effective method based on the CTDE paradigm: Collaborative Policy Learning and Transfer (CPLT). As shown in Fig. 1, CPLT contains the Control Action Generation (CAG) module, which constructs the transferable action-value network to generate the actions of agents, and the Collaborative Control Optimization (CCO) module, which establishes the transferable mixing network to optimize the teamwork. The main contributions of this work are summarized as follows.

- CAG can take advantage of both spatial and temporal observation features to enhance the effectiveness of policy learning, and also can unify the dimension of the input and output data of the action-value network to ensure the cross-scenario transferability of learned policies.
- CCO can generate a consistent-dimensional input for the mixing network in different scenarios by means of Q -value aggregation, and also can prevent unreliable contribution evaluation among the agents based on dynamic masking.
- Experimental results on StarCraft Multi-Agent Challenge (SMAC) and Multi-Agent Particle Environment (MPE) [27,28] demonstrate that CPLT can perform well in both within-scenario and cross-scenario collaborative tasks. Results also validate the compatibility of CAG and CCO with the related value-based approaches to enhance their gaming performances.

2. Preliminary

2.1. Decentralized partially observable Markov decision process

In CMARL, each agent can only observe the local information of the environment and take an action based on its observations. Consequently, a fully cooperative multi-agent problem, where all agents

share a common reward from the environment and strive toward a joint goal, can be modeled as a Decentralized Partially Observable Markov Decision Process (Dec-POMDP) [29].

Definition 1. Dec-POMDP generally consists of a tuple $\langle S, O, Z, A, R, P \rangle$. At each timestep t , the global environment state is denoted as $s^t \in S$, where S is the state set. Each agent i , where $i \in \{1, 2, \dots, N\}$ and N is the number of agents, receives a local observation $o_i^t = O(s^t, i) : S \times \{1, 2, \dots, N\} \rightarrow Z$, and Z is the observation set. Then, each agent takes an action $a_i^t \in A$, and the joint action of all agents is represented as $\mathbf{a}^t = \{a_1^t, \dots, a_N^t\} \in \mathbf{A}$, where A is the action set and $\mathbf{A}$ is the joint action set. Based on the current state and joint action, the environment feeds back a common reward $r_{t+1} = r(s^t, \mathbf{a}^t) : S \times \mathbf{A} \rightarrow R$ to all agents, where R is the reward set. Besides, the state transition function, denoted as $P(s^{t+1} | s^t, \mathbf{a}^t) : S \times \mathbf{A} \times S \rightarrow [0, 1]$, defines the probability of transitioning to state s^{t+1} given the current state s^t and joint action $\mathbf{a}^t$.

2.2. Transformer and attention mechanism

In CAG, the Transformer encoder [30,31], which mainly consists of the self-attention mechanism, Feedforward Neural Network (FNN), and Layer Normalization (LN), is used to construct the action-value network for each agent. For an input sequence X , the Transformer extracts the spatial features X_f by

$$\mathcal{K} = F_{\mathcal{K}}(X), \mathcal{Q} = F_{\mathcal{Q}}(X), \mathcal{V} = F_{\mathcal{V}}(X), \quad (1)$$

$$X_s = \text{Attention}(\mathcal{K}, \mathcal{Q}, \mathcal{V}) = \text{Softmax}\left(\frac{\mathcal{Q}\mathcal{K}^T}{\sqrt{d_k}}\right)\mathcal{V}, \quad (2)$$

$$X_f = \text{LN}(\text{FNN}(\text{LN}(X_s, X))), \quad (3)$$

where $F_{\mathcal{K}}$, $F_{\mathcal{Q}}$ and $F_{\mathcal{V}}$ denote the fully connected layers to calculate the key, query, and value vectors $\mathcal{K}$, $\mathcal{Q}$, and $\mathcal{V}$, respectively; X_s denotes the features extracted by the self-attention mechanism to capture the contextual information from the input sequence; d_k denotes the dimension of key vector; FNN can further enhance the quality of the features extracted by the self-attention layer.

In CCO, the attention mechanism [32,33], which is used to compute the attention weights w_i between the key k_i and the query q_i , estimates the attention weights as the learned relevance scores among agents conditioned on the global state.

$$w_i = \text{Softmax}\left(\frac{q_i k_i^T}{\sqrt{d_k}}\right) = \frac{\exp(\frac{\langle q_i, k_i \rangle}{\|q_i\|_2 \cdot \|k_i\|_2})}{\sum_{j=1}^n \exp(\frac{\langle q_j, k_j \rangle}{\|q_j\|_2 \cdot \|k_j\|_2})}, \quad (4)$$

where w_i is calculated based on the similarity between $q_i \in \mathcal{Q}$ and $k_i \in \mathcal{K}$.

3. Methods

3.1. Local observation decomposition

Generally, in value-based approaches, agents use their local observations of the environment as the input to the action-value network. Then, the CMARL models control every agent to take an action based on its local observation. To tackle the different dimensions of the observation vector across scenarios, we propose a local observation decomposition method.

Specifically, we decompose the local observation o_i^t of each agent into K observation entities $o_{i,k}^t$:

$$o_i^t = \{o_{i,1}^t, o_{i,2}^t, \dots, o_{i,K}^t\}, \quad K = N + N_t, \quad (5)$$

where $o_{i,k}^t$ denotes an element of the local observation, such as the state of an agent or a target unit, and $k \in \{1, \dots, K\}$; N denotes the number of agents empowered by the CMARL model; N_t denotes the number

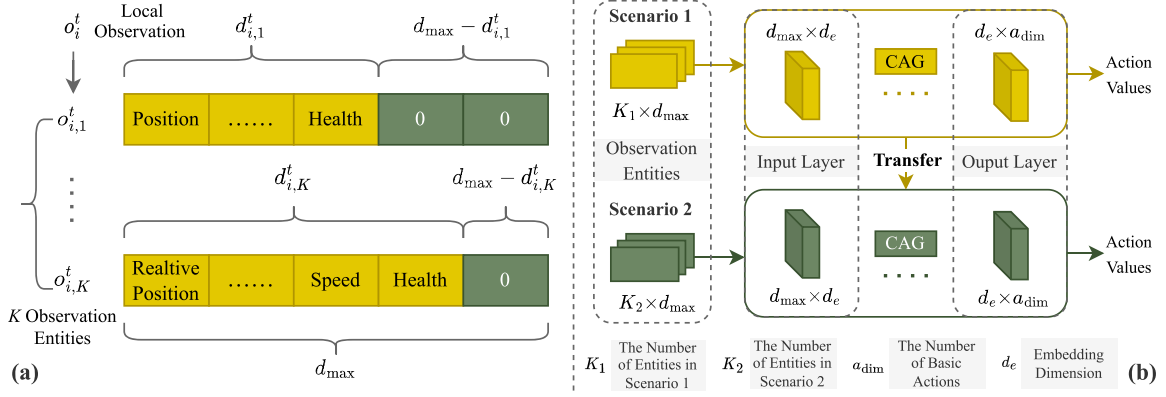


Fig. 2. (a) The decomposition of local observation into entities with zero-padding. (b) The dimensional consistency of input layer in CAG between different scenarios.

of target units (e.g., adversaries) controlled by the built-in script AI of gaming platform.

As shown in Fig. 2(a), each observation entity $o_{i,k}^t \in \mathbb{R}^{1 \times d_{i,k}}$, where $d_{i,k}$ is equal to the number of observation types (e.g., the spatial position and health state of the observed agents or target units) in each entity $o_{i,k}^t$, and all entities can maintain the same dimension via the zero-padding technique:

$$o_{i,k}^t = \begin{cases} \text{Concat}[o_{i,k}^t, 0_{d_{\max}-d_{i,k}}], & \text{if } d_{i,k} < d_{\max}, \\ o_{i,k}^t, & \text{otherwise.} \end{cases} \quad (6)$$

$$d_{\max} = \max(d_{i,k}), i \in \{1, \dots, N\}, k \in \{1, \dots, K\}, \quad (7)$$

where ‘Concat’ stands for the operation of vector concatenation; $d_{\max}$ denotes the maximum value of $d_{i,k}$ among all the observation entities. Therefore, as shown in Fig. 2(b), for the similar tasks in different scenarios, the dimensional consistency of input layer in CAG can be ensured by maintaining the sameness of $d_{\max}$.

3.2. Transferable action-value network

In CAG, we use the observation entities with the same dimension as the input to the action-value network. To handle the varying number of observation entities across scenarios, CAG employs the Transformer blocks to extract the spatial features $\mathcal{F}_o$, considering that the Transformer blocks are compatible with the variable-length inputs and have been widely used in computer vision and natural language processing [30,31,34,35].

$$o_{i,e}^t = \{\mathcal{F}_{\text{encode}}(o_{i,1}^t), \dots, \mathcal{F}_{\text{encode}}(o_{i,K}^t)\}, \quad (8)$$

$$\mathcal{F}_o^l = \mathcal{T}^l(o_{i,e}^t, h_i^{l-1}), \quad (9)$$

where $\mathcal{F}_{\text{encode}}$ denotes the fully connected layer for encoding the observation entities; $l \in \{1, 2, \dots, L\}$ indicates the l th Transformer block; ‘ $\mathcal{T}$ ’ stands for the calculation process of Eq. (1)–(3); $o_{i,e}^t$ denotes the encoded representation; $(o_{i,e}^t, h_i^{l-1})$ denotes the input sequence X ; h_i^{l-1} denotes the hidden state that contains the historical decision-making information at timestep $t-1$.

As shown in Fig. 3(c), since Recurrent Neural Network (RNN) is suitable for temporal modeling and has been widely applied in sequence processing [36–38], CAG further employs RNN, which is composed of Fully Connected layers (FC) and GRU [23], to extract the temporal features $\mathcal{F}_{\text{tem}}$ from the local observation of each agent.

$$\mathcal{F}_{\text{tem}} = \text{GRU}(\text{FC}_1(\mathcal{F}_o)). \quad (10)$$

Next, the individual Q -values of each agent are calculated by the fully connected layer $\text{FC}_2(\mathcal{F}_{\text{tem}})$. Note that the output dimension of FC_2

is determined by the size of the action set of agents. To ensure the consistency of the output dimension of FC_2 between different scenarios, CAG divides the action set A into two subsets A_1 and A_2 . Here, A_1 includes the basic actions (i.e., move, stop, and no-operation), while A_2 involves the interactive actions (i.e., attack and assistance). CAG unifies the output dimension of FC_2 to the fixed number of basic actions in different scenarios. Subsequently, CAG calculates the individual Q -values Q_i by

$$Q_i = \text{Concat}[\text{FC}_2(\mathcal{F}_{\text{tem}, A_1}), \text{Mean}(\text{FC}_2(\mathcal{F}_{\text{tem}, A_2}))], \quad (11)$$

where $\mathcal{F}_{\text{tem}, A_1}$ denotes the features about agent i in $\mathcal{F}_{\text{tem}}$; $\mathcal{F}_{\text{tem}, A_2}$ denotes the features about the other agents and target units in $\mathcal{F}_{\text{tem}}$.

By fully leveraging both spatial and temporal features, CAG can enhance the effectiveness of learned policies in complex multi-agent scenarios. By unifying the dimensions of both input and output data of the action-value network in different scenarios, CAG can ensure the feasibility of cross-scenario policy transfer.

3.3. Transferable mixing network

The different number of agents usually leads to the inconsistent input dimension of the mixing network in value-based CMARL models, which directly limits the model transferability across different scenarios. To cope with it, we design a transferable mixing network in CCO.

Specifically, to handle the inconsistent input dimensions, CCO utilizes the attention mechanism, which has advantage in dynamic weight assignment and has been widely used in pattern recognition [32,33,39], to compute the weight vector W_i for the individual Q -values of each agent. As shown in Fig. 3(a), CCO selects the self-observation entity $o_{i,1}^t$ of each agent based on Eq. (5), and utilizes it to construct a self-observation vector o_{self} as the input of attention mechanism:

$$o_{\text{self}} = [o_{1,1}^t, o_{2,1}^t, \dots, o_{N,1}^t]. \quad (12)$$

Next, CCO treats the global environment state s^t as the query and the self-observation vector o_{self} as the key in the attention mechanism. By calculating the similarity between s^t and o_{self} , CCO outputs a weight vector W^t according to Eq. (4). Here, W^t contains the estimated contribution weight w_i^t of each agent:

$$W^t = [w_1^t, w_2^t, \dots, w_N^t] = \text{Softmax}\left(\frac{s^t \cdot o_{\text{self}}^T}{\sqrt{d_k}}\right). \quad (13)$$

To prevent the unreliable contribution calculation, CCO leverages a dynamic mask vector D_{mask}^t to set the weights of inactive agents as zero in W^t :

$$D_{\text{mask}}^t = [D_1^t, D_2^t, \dots, D_N^t], \quad D_i^t = 1 \text{ or } 0, \quad (14)$$

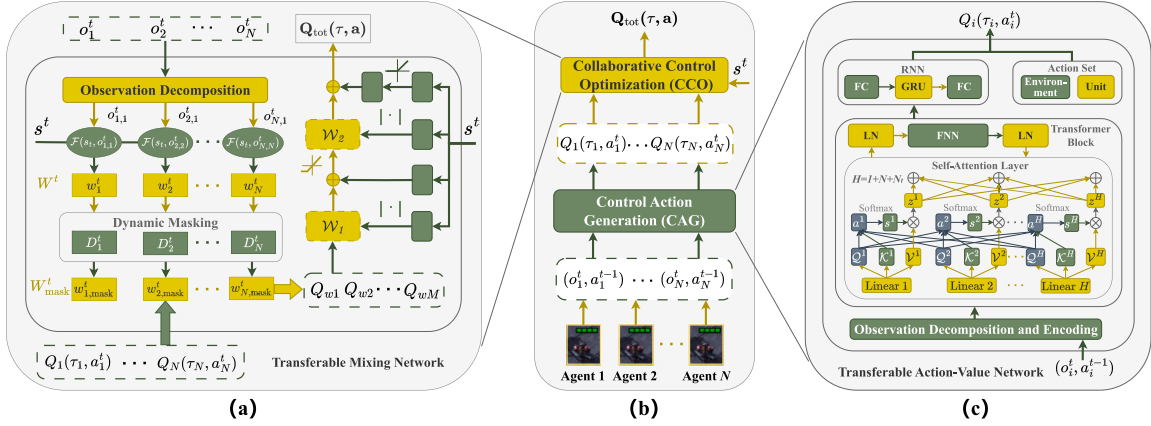


Fig. 3. (a) CCO takes advantage of Q -value aggregation and dynamic masking to weight the individual Q -values of agents, and then calculates the joint Q -value through the hypernetworks; (b) the framework of CPLT; (c) CAG extracts the features from the local observation of each agent, and then establishes the mapping between the observation features and action sets to calculate the individual Q -values for all agents.

$$W_{mask}^t = (D_{mask}^t)^T \cdot W^t = [w_1^t D_1^t, \dots, w_N^t D_N^t] \\ = [w_{1,mask}^t, w_{2,mask}^t, \dots, w_{N,mask}^t], \quad (15)$$

where the mask D_i^t is set as 1 for the active agents and 0 for the inactive ones; W_{mask}^t denotes the masked weight vector.

Then, CCO uses the masked weight vector W_{mask}^t to project the individual Q -values of agents into the aggregated Q -value Q_w of active agents:

$$Q_w(\tau, a) = \sum_{i=1}^N w_{i,mask}^t \cdot Q_i(\tau_i, a_i^t), \quad (16)$$

where τ_i indicates the action-observation history of agent i .

The input dimension of the mixing network varies across scenarios due to the change in the number of agents. To maintain the input dimension consistency across scenarios and enhance the expressive power of the mixing network, we introduce the value entity Q_{wm} , where $Q_{wm} = Q_w$ for which $m \in \{1, 2, \dots, M\}$, into CCO. CCO uses this value entity to unify the input dimension of the mixing network to a fixed dimension, and then calculates the joint Q -value Q_{tot} based on the value entity Q_{wm} :

$$Q_{tot}(\tau, a, s) = \mathcal{W}_2 \cdot \left(\sum_{m=1}^M \mathcal{W}_1 \cdot Q_{wm} + b_1 \right) + b_2, \quad (17)$$

where τ represents the joint action-observation history of all agents; $\mathcal{W}_1$, $\mathcal{W}_2$ and b_1 , b_2 denote the weights and biases, respectively, which are generated from the hypernetworks mainly consisting of linear layers; M denotes the unified input dimension of mixing network, which stays within the interval $[1, N + 1]$.

By ensuring the consistency of input dimensions in the mixing network, the pre-trained model of CCO can be transferred to new scenarios. By addressing the problem of unreliable contribution evaluation caused by the inactive agents through dynamic masking, CCO enables more effective exploration of the complex relationship between Q_{tot} and Q_i to optimize collaborative teamwork.

3.4. Training of CPLT

With the individual Q -values calculated by Eq. (11), at timestep t , each agent can take an action a_i^t based on the ϵ -greedy policy. More concretely, each agent either takes the random action with the probability of ϵ or performs the action with the highest Q_i value:

$$a_i^t = \begin{cases} \text{random}(A), & \text{if } \rho < \epsilon, \\ \arg \max_a Q_i(\tau_i, a), & \text{otherwise,} \end{cases} \quad (18)$$

where $\text{random}(A)$ indicates an action randomly selected from the action set A , and $\rho \in [0, 1)$ denotes a randomly-generated probability.

Further, to optimize the entire CPLT network, the loss function $\mathcal{L}$ is defined based on the Temporal Difference (TD) error [40]:

$$\mathcal{L} = \sum_{j=1}^b \left(r_j + \gamma \max_{a'} \bar{Q}_{j,tot}(\tau', a') - Q_{j,tot}(\tau, a) \right)^2, \quad (19)$$

where Q_{tot} denotes the joint Q -value output from the mixing network, while $\bar{Q}_{tot}$ denotes the target joint Q -value generated by the target network of mixing network. The variable b is the size of the batches sampled from the replay buffer. The training procedures of CPLT have been displayed in Algorithm 1.

To sufficiently reuse the learned experience in different scenarios, the pre-trained CPLT model can be transferred from the old scenario to a new one. Moreover, it is feasible to change the action-value network of the related value-based methods to CAG, or replace the mixing network of them by CCO, to ensure their policy learning effectiveness and cross-scenario transferability.

Algorithm 1 Training Procedures of CPLT

Initialize the experimental map, the parameters ϑ of action-value and mixing networks, the parameters $\bar{\vartheta}$ of target network, and the maximum timestep T .

for each training episode do

for $t = 1$ to T **do**

for each agent do

Calculate the individual Q -values using Eq. (11);

Select an action a_i^t using Eq. (18);

end

Take the joint action $\mathbf{a}^t = (a_1^t, \dots, a_N^t)$ at environment state s^t ;

Acquire the reward r_{t+1} and new environment state s^{t+1} ;

Store $(s^t, \mathbf{a}^t, r_{t+1}, s^{t+1})$ into the replay buffer;

end

Sample b episodes from the replay buffer;

for episode $= 1$ to b **do**

Calculate the joint Q -value using Eq. (17);

Calculate the loss $\mathcal{L}$ using Eq. (19);

end

Update ϑ by minimizing the loss $\mathcal{L}$;

Update $\bar{\vartheta} = \vartheta$ every 200 timesteps.

end

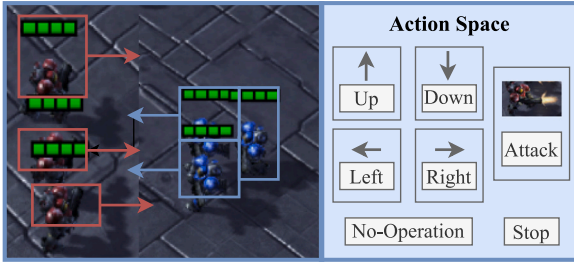


Fig. 4. On SMAC, red units indicate the agents controlled by the CMARL algorithms, while blue units indicate the adversaries controlled by built-in script AI in the game environment. Each agent can take both basic and interactive actions.

3.5. Modeling rationality of CPLT

The modeling rationality of CPLT can be interpreted from the following three aspects. Firstly, based on the Transformer-RNN architecture, CAG not only can fully utilize the spatio-temporal observation features for effective policy generation, but also can unify both the input and output dimensions of action-value network in different scenarios. Secondly, attributing to the attention mechanism and dynamic masking, CCO can avoid the unreliable contribution evaluation for effective policy optimization and unify the input dimension of mixing network in different scenarios. Thirdly, according to the derivation in Eq. (20), CPLT satisfies the Individual Global Maximum (IGM) condition [22,41,42], which ensures that the joint actions optimized by CCO are equivalent to the set of individual actions produced by CAG. Specifically, according to Eq. (16)–(17), the partial derivative $\partial Q_{\text{tot}}/\partial Q_i$ is formulated as

$$\begin{aligned} \frac{\partial Q_{\text{tot}}}{\partial Q_i} &= \frac{\partial \left[\mathcal{W}_2 \cdot \left(\sum_{m=1}^M \mathcal{W}_1 \cdot Q_{iwm} + b_1 \right) + b_2 \right]}{\partial Q_i} \\ &= \frac{\partial \left[\mathcal{W}_2 \cdot \left(\sum_{m=1}^M \mathcal{W}_1 \cdot \sum_{i=1}^N w'_{i, \text{mask}} \cdot Q_i + b_1 \right) + b_2 \right]}{\partial Q_i} \\ &= \mathcal{W}_2 \cdot \sum_{m=1}^M \mathcal{W}_1 \cdot \sum_{i=1}^N w'_{i, \text{mask}}, \end{aligned} \quad (20)$$

where $\mathcal{W}_1$ and $\mathcal{W}_2$ are the non-negative weights generated by the hypernetworks, and $w'_{i, \text{mask}}$ is calculated by Eq. (13)–(15). So, $\mathcal{W}_1$, $\mathcal{W}_2$ and $w'_{i, \text{mask}}$ are not less than 0, which means the non-negativity of Eq. (20), i.e. $\partial Q_{\text{tot}}/\partial Q_i \geq 0$. Since $\partial Q_{\text{tot}}/\partial Q_i \geq 0$ is the sufficient condition of IGM [22,41,42], the joint actions optimized by CCO are equivalent to the set of individual actions produced by CAG in CPLT.

4. Experiments

4.1. Experimental setup

We conduct our experiments on SMAC [27], a widely adopted benchmark platform based on the real-time strategy game StarCraft II, as illustrated in Fig. 4. On SMAC, each unit has a limited observation range, which requires agents to make decisions based on the local information of the environment. The goal of CMARL is to coordinate a group of agents to defeat the target units (i.e., adversaries) controlled by built-in script AI. A victory is achieved by annihilating all the adversaries within a predefined time limit. On MPE [28], we select a cooperative navigation task, which controls N agents to collaborate to cover N landmarks in a two-dimensional environment, for cross-scenario experiments.

We not only evaluate the performance of CPLT in a single game scenario, but also verify its transferability across different game scenarios. The details of all the experimental maps are presented in Table 1. In

Table 1
Map Details of SMAC and MPE.

Platform	Map Names	Types
SMAC	8m_vs_9m	8 Marines vs 9 Marines
	3s_vs_3z	3 Stalkers vs 3 Zealots
	5m_vs_6m	5 Marines vs 6 Marines
	3s_vs_5z	3 Stalkers vs 5 Zealots
	3s5z	3 Stalkers 5 Zealots vs 3 Stalkers 5 Zealots
	3s5z_vs_3s6z	3 Stalkers 5 Zealots vs 3 Stalkers 6 Zealots
MPE	Cooperative Navigation 3	3 Agents and 3 Landmarks
	Cooperative Navigation 4	4 Agents and 4 Landmarks

Table 2
Key Hyperparameter Settings for Method Training.

Hyperparameters (All Evaluated Methods)	Values
Batch Size	32
Buffer Size	5000
Discount Factor	0.99
RMSProp Learning Rate	0.0005
Target Network Update Interval	200
Hypernetwork Embedding Dimension	64
Hyperparameters (CPLT and UPDeT)	Values
Transformer Embedding Dimension	32
Transformer Head Number	3
Transformer Layer Number	2

our experiments, each method is evaluated through five independent training runs to reduce result randomness. In the training process, we evaluate the performance of each method over 32 test episodes after every 10,000 training steps. The test win rate, defined as the proportion of winning episodes to total test episodes, serves as the key metric for evaluating the effectiveness of the model. Besides, the performances of CPLT and all compared methods are evaluated based on the PC with the GPU of NVIDIA GeForce RTX 3090 Ti and the CPU of Intel Core i7-12700F. The hyperparameter settings for method training are shown in Table 2.

4.2. Experimental results

At first, to demonstrate the effectiveness of CPLT in the single game scenario, we compare the proposed method not only with the typical value-based approaches QMIX [22], MCMARL [17], and QPLEX [16], but also with the transferable CMARL approaches UPDeT [24] and TransQMIX [25]. As shown in Fig. 5, CPLT outperforms all compared methods in terms of the final test win rate and convergence speed. On the hard map 3s_vs_5z and the super hard map 3s5z_vs_3s6z, the differences in the number and types between agents and target units increase the difficulty of policy learning. Consequently, compared to the easy maps 3s_vs_3z and 8m_vs_9m, CPLT exhibits a decrease in algorithm stability and final test win rate, but it still outperforms the compared methods.

Next, to evaluate the computational complexity of CAG and CCO in CPLT, we have reported the parameter amount, Multiply-Accumulate Operations (MACs), and training time of CPLT, QMIX and UPDeT. As recorded in Table 3, we present the parameter amount and MACs for both action-value network and mixing network in CPLT, QMIX and UPDeT. Attributing to the representative spatio-temporal features extracted by the Transformer and RNN in CAG and owing to the precise contributions calculated by the attention mechanism with dynamic masking in CCO, both the MACs and training time of CPLT are higher than those of QMIX and UPDeT. Although the modules of CAG and CCO in CPLT incur the higher computational complexity, CPLT can achieve the higher win rates with smaller confidence intervals in both within-scenario and cross-scenario collaborative tasks. Actually, with the swift development of modern hardwares such as GPU and CPU, the training time of CPLT will hopefully be much reduced in the not-so-far future.

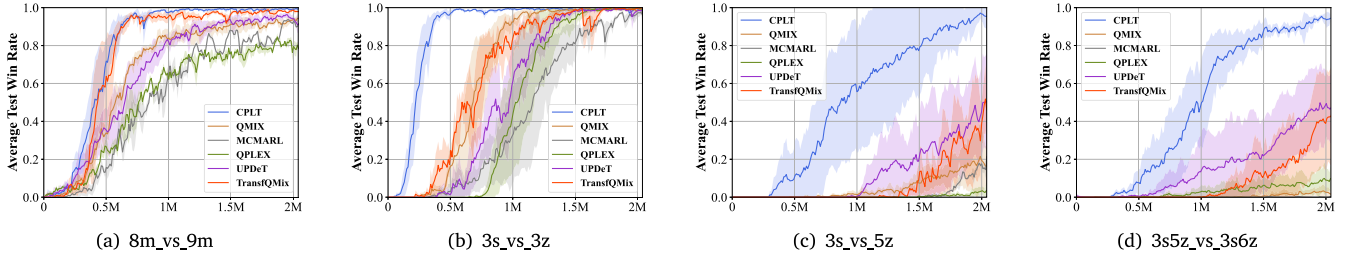


Fig. 5. Single-scenario experiments: each curve represents the average test win rate, while the shaded area indicates the standard deviation interval of each method. A smaller interval suggests greater method stability.

Table 3

Parameter Amount, Multiply-Accumulate Operations (MACs), and Time Cost for Method Training.

Methods	8m_vs_9m					3s5z_vs_3s6z				
	Action-Value Network		Mixing Network		Whole Model	Action-Value Network		Mixing Network		Whole Model
	Parameter Amount	MACs	Parameter Amount	MACs	Time Cost	Parameter Amount	MACs	Parameter Amount	MACs	Time Cost
QMIX	31.44k	8.14M	53.31k	1.69M	3h2m	34.71k	8.98M	63.11k	2.01M	3h9m
UPDeT	43.05k	197.01M	53.31k	1.69M	7h5m	43.14k	197.43M	63.11k	2.01M	6h27m
CPLT	49.38k	198.69M	66.45k	69.85M	7h10m	49.48k	199.11M	64.07k	67.84M	6h38m

Abbreviations: k - kilobytes; M - millions; h - hours; m - minutes.

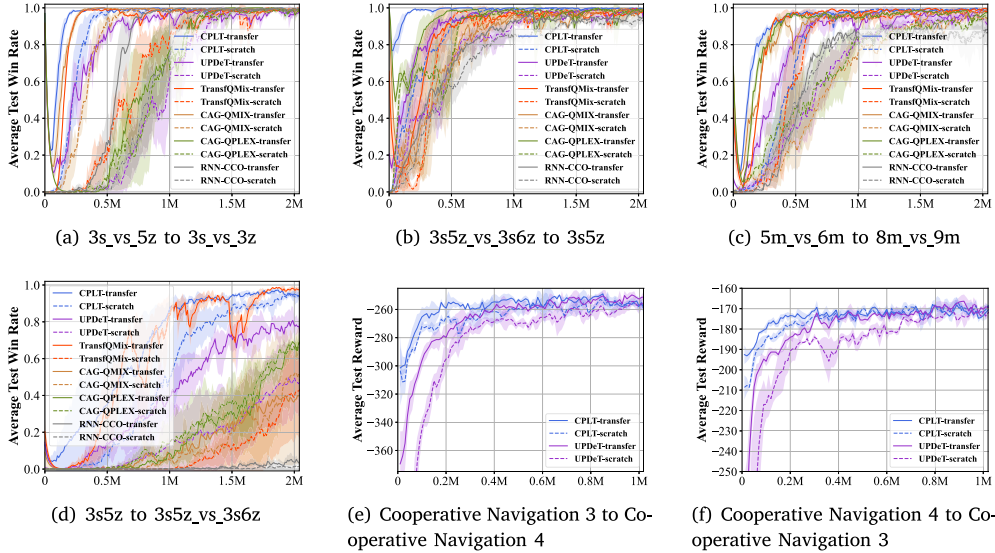


Fig. 6. Cross-scenario experiments: the methods are pre-trained in one game scenario and then transferred to another.

Then, to validate the transferability of CPLT across different game scenarios on SMAC, we pre-train it in a scenario and transfer it to another new one, in comparison with UPDeT and TransQMIX. We use the subscript ‘-transfer’ to denote the methods pre-trained for transferring and ‘-scratch’ to denote those trained from scratch. As exhibited in Fig. 6 and Table 4, although the composition of agents and target units differs between the new and old maps, CPLT-transfer can achieve the highest test win rates with the smallest confidence intervals on the new maps. Such results readily demonstrate that CPLT can effectively adapt its learned policies to the new scenarios. To evaluate the transferability of CPLT on MPE, we pre-train CPLT and UPDeT in one cooperative navigation scenario at first, and then transfer the pre-trained models to the other. As exhibited in Fig. 6(e)–(f), CPLT-transfer can effectively adapt the learned collaborative policies to the new scenarios and still outperforms UPDeT-transfer on the whole. Besides, we carry out visual analysis to verify the effectiveness of policy transfer. We choose the maps 5m_vs_6 m and 8m_vs_9 m to exemplify the policy learning and transference process of CPLT, without loss of generality. As shown in Fig. 7(a), the process by which CPLT controls the agents to defeat all

the adversaries can be divided into five main stages. When the pre-trained model of CPLT is transferred to the map 8m_vs_9 m, CPLT can well accommodate the new scenario, and, at the same time, exhibits the five stages similar to those on 5m_vs_6 m. This phenomenon intuitively verifies the strong cross-scenario transferability of CPLT.

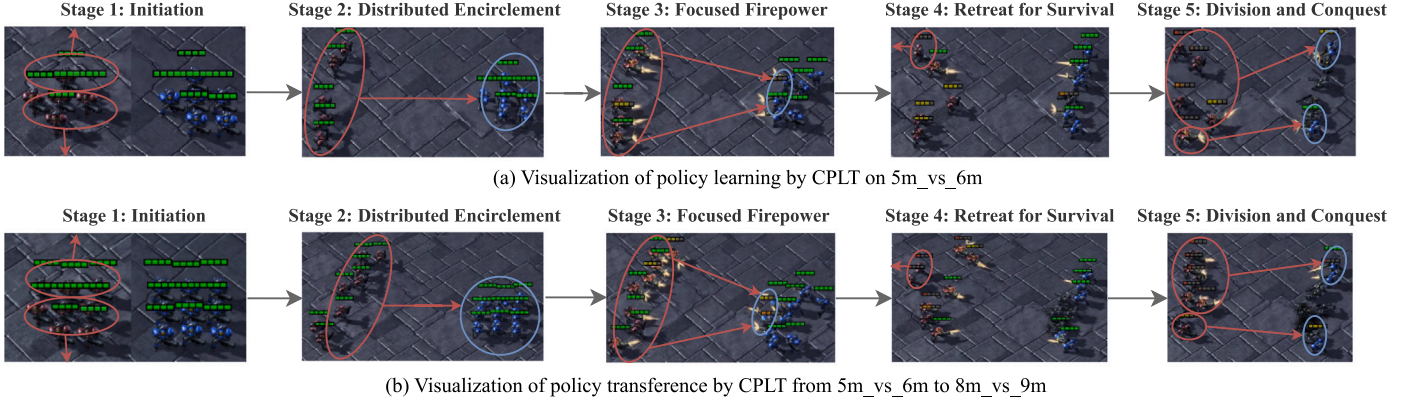
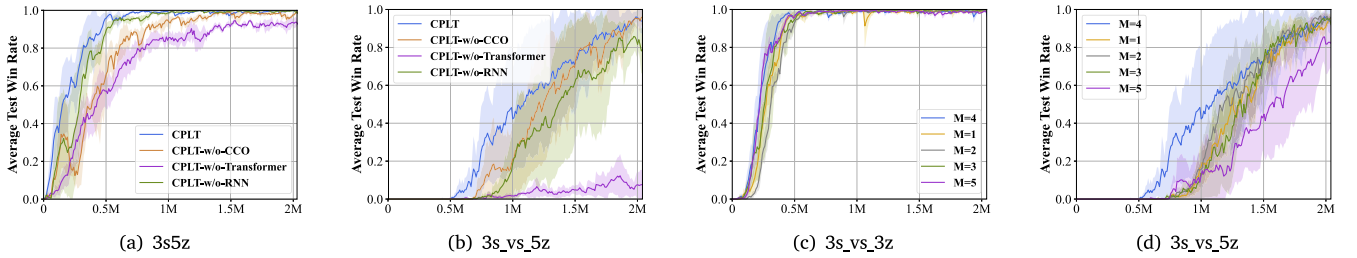
Subsequently, we assess the compatibility of CAG and CCO with the related value-based approaches. Specifically, we evaluate the ability of CAG as the action-value networks of QPLEX and QMIX, and the ability of CCO as the mixing network of QMIX, for both within-scenario and cross-scenario collaborative tasks. For clarity, we denote the models trained from scratch as ‘x-x-scratch’ and those pre-trained for transferring as ‘x-x-transfer’. As reflected in Fig. 6 and Table 4, compared with the original QMIX and QPLEX models in Fig. 5, the integration of either CAG or CCO with QMIX and QPLEX consistently results in the higher test win rates with the smaller confidence intervals. These results demonstrate that integrating CAG or CCO with the related value-based approaches can enhance their gaming performances in both within-scenario and cross-scenario collaborative tasks.

Table 4

The Best Test Win Rates with the Corresponding Confidence Intervals of All Evaluated Methods in Fig. 6(c)-(d).

Methods	5m_vs_6m to 8m_vs_9m											
	CPLT		UPDeT		TransfQMIX		CAG-QMIX		CAG-QPLEX		RNN-CCO	
	-transfer	-scratch	-transfer	-scratch	-transfer	-scratch	-transfer	-scratch	-transfer	-scratch	-transfer	-scratch
Best Test Win Rate	99.87	99.62	98.58	96.29	99.78	98.84	98.85	96.27	98.84	95.69	91.64	88.36
Confidence Interval	0.02	0.39	0.68	1.73	0.05	1.21	0.07	2.19	0.18	1.75	0.99	5.98

Methods	3s5z to 3s5z_vs_3s6z											
	CPLT		UPDeT		TransfQMIX		CAG-QMIX		CAG-QPLEX		RNN-CCO	
	-transfer	-scratch	-transfer	-scratch	-transfer	-scratch	-transfer	-scratch	-transfer	-scratch	-transfer	-scratch
Best Test Win Rate	97.35	95.28	80.76	48.84	97.31	44.12	40.01	52.43	69.25	69.27	5.31	1.82
Confidence Interval	1.02	0.91	7.61	21.3	1.89	27.8	37.1	13.1	0.54	3.76	2.25	0.53

**Fig. 7.** Visualization of the policy learning and transference of CPLT. The red arrow refers to the intention of agents for moving or attacking. (a) Visualization of policy learning by CPLT on 5m_vs_6 m. (b) Visualization of policy transference by CPLT from 5m_vs_6 m to 8m_vs_9 m.**Fig. 8.** Model ablation of CPLT on 3s5z and 3s_vs_5z, as well as hyperparameter evaluation of M for CPLT on 3s_vs_3z and 3s_vs_5z.

After that, to justify the effectiveness of CCO in CPLT, we replace it with the hypernetworks used in QMIX, denoted as CPLT-w/o-CCO. As shown in Fig. 8(a)–(b), CPLT performs better than CPLT-w/o-CCO and confirms the usefulness of CCO in CPLT. To verify the effectiveness of CAG in CPLT, we remove the Transformer in CAG, denoted as CPLT-w/o-Transformer. By comparing CPLT-w/o-Transformer with CPLT, we find that the decision-making effectiveness of agents decreases when the action-value network misses the spatial information, which is extracted from the observation entities via the Transformer. Moreover, we remove the RNN module in CAG, denoted as CPLT-w/o-RNN. Compared with CPLT, CPLT-w/o-RNN incurs a greater number of convergence steps due to the absence of temporal information, which hinders the effective utilization of historical decision information.

At last, we analyze the impact of the hyperparameter M , which defines the input dimension of CCO, on the performance of CPLT. If M is too small, it tends to limit the expressive power of the CCO network, so that it will be hard for CCO to effectively explore the complex relationship between Q_{tot} and Q_i for policy optimization. Conversely, if M is too large, the CCO network is prone to overfitting in the training phase, which limits the testing performance of CPLT. From Fig.

8(c)–(d), CPLT achieves optimal performance in single scenarios when $M = 4$. Besides, we pre-train the CPLT model with different settings of M on 3s_vs_5z, and transfer the pre-trained model to 3s_vs_3z. As visualized in Fig. 9, we use the bar chart to showcase the initial test win rates, the best test win rates and the corresponding confidence intervals of the proposed model under different M settings. From the figure, we notice that although CPLT eventually achieves the similar performances with different M values, it acquires the relatively highest initial test win rate with the smallest confidence interval when $M = 4$. Hence, we recommend setting $M = 4$ for our method in experiments by default.

5. Conclusion

In this paper, we have proposed CPLT, a new CMARL framework for learning and transferring collaborative control policies among multiple agents in the real-time strategy game environment. With the designed CAG and CCO modules, our method can enable the agents to learn effective policies from their local observations and adapt their behaviors to the different scenarios. Experimental results on the SMAC platform have demonstrated the effectiveness and transferability of CPLT for

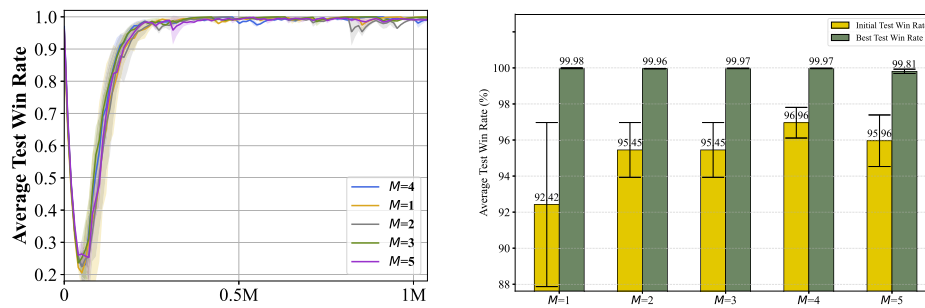


Fig. 9. The performances of CPLT transferred from 3s_vs_5z to 3s_vs_3z under different M settings.

agents in the game environment, and also validated the capability and compatibility of CAG and CCO in cooperation with the related value-based approaches to enhance their gaming performances. These results show that our method is suitable and capable of enhancing the collaborative behaviors of agents in the game environment. So, this work will hopefully benefit the prospective development of adaptive game AI and multi-agent entertainment systems. Nevertheless, due to the significant platform difference, it is difficult for CPLT to effectively transfer the learned policies across different platforms. Meanwhile, as a value-based CMARL method, CPLT suffers from the limitation to handle the tasks that rely on policy learning in a continuous action space. To overcome this limitation, we will tentatively remodel CPLT in a policy-based architecture in future.

CRedit authorship contribution statement

Wei Li: Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Software, Resources, Project administration, Methodology, Investigation, Funding acquisition, Formal analysis, Data curation, Conceptualization. **Jiali Lv:** Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Resources, Methodology, Investigation, Formal analysis, Data curation. **Xu Zhang:** Project administration, Resources, Investigation, Writing – review & editing. **Kaizhu Huang:** Formal analysis, Investigation, Resources, Writing – review & editing. **Aiguo Song:** Project administration, Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported by the Open Fund of National Key Laboratory of Avionics Integration and Aviation System-of-Systems Synthesis [Grant Number: 2025AIIASS0102] and supported by the Open Fund of Key Laboratory of the Ministry of Education on Artificial Intelligence in Equipment [Grant Number: 2024-AAIE-KF03-01].

Data availability

Data will be made available on request.

References

- [1] Z. Liu, J. Zhang, Z. Liu, et al., Joint cooperative clustering and power control for energy-efficient cell-free XL-MIMO with multi-agent reinforcement learning, *IEEE Trans. Commun.* 72 (12) (2024) 7772–7786.
- [2] N. Chen, Entertainment robots based on swarm intelligence algorithm applied in remote dance performances, *Entertain. Comput.* 52 (2025) 100783(1)–100783(7).
- [3] J. Wu, C. Huang, H. Huang, et al., Recent advances in reinforcement learning-based autonomous driving behavior planning: A survey, *Transp. Res. Part C Emerg. Technol.* 164 (2024) 104654(1)–104654(23).
- [4] Q. Liang, J. Liu, Z. Jiang, et al., Limited information aggregation for collaborative driving in multi-agent autonomous vehicles, *IEEE Rob. Autom. Lett.* 9 (7) (2024) 6624–6631.
- [5] C. Sun, Z. Zang, J. Li, et al., T2MAC: targeted and trusted multi-agent communication through selective engagement and evidence-driven integration, in: *AAAI Conf. Artif. Intell.*, AAAI, Vancouver, British Columbia, 2024, pp. 15154–15163.
- [6] J. Liu, S. Ding, BI in simulation analysis with gaming for decision making and development of knowledge management, *Entertain. Comput.* 52 (2025) 100811(1)–100811(9).
- [7] R.G. Barbosa, M.A.F. Rodrigues, An interchangeable editor to create generic and adaptable decision trees for versatile applications and game development scenarios, *Entertain. Comput.* 52 (2025) 100864(1)–100864(14).
- [8] J. Chen, P. Chen, X. Niu, et al., Task offloading in hybrid-decision-based multi-cloud computing network: a cooperative multi-agent deep reinforcement learning, *J. Cloud Comput.* 11 (1) (2022) 90–107.
- [9] J. Baek, G. Kaddoum, FLoadNet: Load balancing in fog networks with cooperative multiagent using actor-critic method, *IEEE Trans. Netw. Serv. Manag.* 20 (1) (2022) 400–414.
- [10] S. Menon, S.R. Addula, A. Parkavi, et al., Streamlining task planning systems for improved enactment in contemporary computing surroundings, *SN Comput. Sci.* 5 (993) (2024) 1–9.
- [11] X. Lan, J. Yan, S. He, et al., Distributed cooperative reinforcement learning for multi-agent system with collision avoidance, *Internat. J. Robust Nonlinear Control* 34 (1) (2024) 567–585.
- [12] H. Wu, A. Ghadami, B.I. Epureanu, Dynamic task planning for autonomous reconfigurable manufacturing systems by knowledge-based multi-agent reinforcement learning, *CIRP Ann.* 73 (1) (2024) 353–356.
- [13] S.R. Addula, Y. Ramaswamy, D. Dawadi, et al., Blockchain-enabled healthcare optimization: Enhancing security and decision-making using the mother optimization algorithm, in: *Int. Conf. Intell. Cloud Comput.*, IEEE, Bhubaneswar, India, 2025, pp. 1–8.
- [14] S.R. Addula, M.K. Kumar, D. Kumar, et al., Dynamic load balancing in cloud computing using hybrid kookaburra-pelican optimization algorithms, in: *Int. Conf. Augm. Real., Intell. Syst., Indus. Aut.*, IEEE, Manipal, India, 2024, pp. 1–7.
- [15] X. Liu, B. Jin, Information-theoretic multi-agent algorithm based on the CTDE framework, in: *Int. Conf. Elec. Tech. Inf. Scien.*, IEEE, Hangzhou, China, 2024, pp. 511–516.
- [16] J. Wang, Z. Ren, T. Liu, et al., QPLEX: Duplex dueling multi-agent Q-learning, 2021, pp. 1–27, arXiv:2008.01062.
- [17] J. Zhao, M. Yang, Y. Zhao, et al., MCMARL: Parameterizing value function via mixture of categorical distributions for multi-agent reinforcement learning, *IEEE Trans. Games* 16 (3) (2024) 556–565.
- [18] Z. Chen, B. Luo, T. Hu, X. Xu, LJIR: Learning Joint-Action Intrinsic Reward in cooperative multi-agent reinforcement learning, *Neural Netw.* 167 (1) (2023) 450–459.
- [19] J. Foerster, G. Farquhar, T. Afouras, et al., Counterfactual multi-agent policy gradients, in: *AAAI Conf. Art. Intell.*, AAAI, New Orleans, United States, 2018, pp. 1–9.
- [20] R. Pina, V.D. Silva, J. Hook, A. Kondoz, Residual Q-networks for value function factorizing in multiagent reinforcement learning, *IEEE Trans. Neural Netw. Learn. Syst.* 35 (2) (2024) 1534–1544.

- [21] X. Hu, P. Guo, Y. Li, et al., TVDO: Tchebycheff value-decomposition optimization for multiagent reinforcement learning, *IEEE Trans. Neural Networks Learn. Syst.* 36 (7) (2024) 12521–12534.
- [22] T. Rashid, M. Samvelyan, C.S. De Witt, et al., Monotonic value function factorisation for deep multi-agent reinforcement learning, *J. Mach. Learn. Res.* 21 (1) (2020) 7234–7284.
- [23] G. Shen, Q. Tan, H. Zhang, et al., Deep learning with gated recurrent unit networks for financial sequence predictions, *Procedia Comput. Sci.* 131 (2018) 895–903.
- [24] S. Hu, F. Zhu, X. Chang, X. Liang, UPDeT: Universal Multi-agent reinforcement learning via Policy Decoupling with Transformers, in: *Int. Conf. Learn. Rep., ICLR, Virtual*, 2021, pp. 2025–2039.
- [25] M. Gallici, M. Martin, I. Masmitja, TransfQMix: Transformers for leveraging the graph structure of multi-agent reinforcement learning problems, in: *Auto. Agent. Multi. Syst., AAMAS, London, United Kingdom*, 2023, pp. 1679–1687.
- [26] J. Chai, W. Li, Y. Zhu, et al., UNMAS: Multi-agent reinforcement learning for unshaped cooperative scenarios, *IEEE Trans. Neural Netw. Learn. Syst.* 34 (4) (2021) 2093–2104.
- [27] M. Samvelyan, T. Rashid, C.S. De Witt, et al., The StarCraft multi-agent challenge, 2019, pp. 1–14, [arXiv:1902.04043](https://arxiv.org/abs/1902.04043).
- [28] I. Mordatch, P. Abbeel, Emergence of grounded compositional language in multi-agent populations, in: *AAAI Conf. Arti. Intell., AAAI, New Orleans, Louisiana, United States*, 2018, pp. 1495–1502.
- [29] M. Kayaalp, F. Ghadieh, A.H. Sayed, Policy evaluation in decentralized POMDPs with belief sharing, *IEEE Open J. Control. Syst.* 2 (2023) 125–145.
- [30] K. Han, Y. Wang, H. Chen, et al., A survey on vision transformer, *IEEE Trans. Pattern Anal. Mach. Intell.* 45 (1) (2023) 87–110.
- [31] S.R. Dubey, S.K. Singh, Transformer-based generative adversarial networks in computer vision: A comprehensive survey, *IEEE Trans. Artif. Intell.* 5 (10) (2024) 4851–4867.
- [32] M.H. Guo, T.X. Xu, J.J. Liu, et al., Attention mechanisms in computer vision: A survey, *Comput. Vis. Media* 8 (3) (2022) 331–368.
- [33] S. Liu, Z. Wang, Y. An, et al., EEG emotion recognition based on the attention mechanism and pre-trained convolution capsule network, *Knowledge-Based Syst.* 265 (2023) 110372(1)–110372(11).
- [34] T. Wolf, L. Debut, V. Sanh, et al., Transformers: State-of-the-art natural language processing, in: *Conf. Empir. Meth. Natur. Lang. Process.: Syst. Demonstrat., Association for Computational Linguistics*, 2020, pp. 38–45, Online.
- [35] G. Tucudean, M. Bucos, B. Dragulescu, et al., Natural language processing with transformers: a review, *PeerJ Comput. Sci.* 10 (2024) e2222(1)–e2222(22).
- [36] H. Phan, F. Andreotti, N. Cooray, et al., SeqSleepNet: End-to-end hierarchical recurrent neural network for sequence-to-sequence automatic sleep staging, *IEEE Trans. Neural Syst. Rehabil. Eng.* 27 (3) (2019) 400–410.
- [37] H. Niu, H. Li, J. Wang, et al., Enhancing computer digital signal processing through the utilization of rnn sequence algorithms, *Int. J. Comp. Sci. Inf. Tech.* 1 (1) (2023) 60–68.
- [38] C. Rota, M. Buzzelli, S. Bianco, R. Schettini, A RNN for temporal consistency in low-light videos enhanced by single-frame methods, *IEEE Signal Process. Lett.* 31 (2024) 2795–2799.
- [39] C. Wang, H. Wang, Cascaded feature fusion with multi-level self-attention mechanism for object detection, *Pattern Recognit.* 138 (2023) 109377(1)–109377(10).
- [40] R. Wang, X. Hao, Y. Feng, et al., MA-TDMPC: Multi-agent temporal difference for model predictive control, in: *Conf. Mach. Learn., Cloud Comp. Intell. Min., IEEE, Jiuzhaigou, China*, 2023, pp. 256–266.
- [41] J. Chai, W. Li, Y. Zhu, et al., UNMAS: Multi-agent reinforcement learning for unshaped cooperative scenarios, *IEEE Trans. Neural Netw. Learn. Syst.* 34 (4) (2021) 2093–2104.
- [42] H. ang, C. Wang, S. Chang, J. Zhang, Priority Over Quantity: A self-incentive credit assignment scheme for cooperative multiagent reinforcement learning, *IEEE Trans. Comput. Soc. Syst.* 11 (6) (2024) 7766–7777.